# 🔗 Live Cloud Deployment Link

**Live demo:** `PASTE_YOUR_TITANIC_STREAMLIT_LINK_HERE`

---

**Course:** Machine Learning Fundamentals (AIC354) — COMSATS University Islamabad, Lahore Campus
**Assignment 2, Task 1:** Titanic Passenger Survival Prediction System
**Student:** Mishal Fatima | **Reg. No:** SP24-BCS-067 | **Section:** B
**Instructor:** Dr. Rao Muhammad Adeel Nawab

# Titanic Passenger Survival Prediction System

## Setup: Running This Notebook From Scratch

Follow these steps in a terminal, inside this project's folder, before opening the notebook.

### 1. Check Python is installed

```
python --version
```

This notebook was built and tested on **Python 3.10**. Any Python 3.9-3.12 should work.

### 2. Create a virtual environment

A virtual environment keeps this project's packages separate from the rest of your system.

**Windows (Command Prompt / PowerShell):**

```
python -m venv venv
```

**macOS / Linux:**

```
python3 -m venv venv
```

### 3. Activate the virtual environment

You must do this every time you open a new terminal to work on this project.

**Windows (Command Prompt):**

```
venv\Scripts\activate.bat
```

**Windows (PowerShell):**

```
venv\Scripts\Activate.ps1
```

*(If PowerShell blocks this with an execution-policy error, run once: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`)*

**macOS / Linux:**

```
source venv/bin/activate
```

Your terminal prompt should now show `(venv)` at the start of the line.

### 4. Install the required packages

This project ships a `requirements.txt` file listing the exact package versions it was built and tested with.

```
pip install -r requirements.txt
```

### 5. Register the virtual environment as a Jupyter kernel (recommended)

```
python -m ipykernel install --user --name=titanic-venv --display-name "Python
(titanic-venv)"
```

When you open the notebook, make sure **Kernel -> Change Kernel -> Python (titanic-venv)** is selected, so it uses the packages you just installed.

### 6. Launch Jupyter Notebook

```
jupyter notebook
```

This opens Jupyter in your browser. Click this `.ipynb` file to open it, then run all cells from the top (**Kernel -> Restart & Run All**).

### 7. When you're done

Deactivate the virtual environment with:

```
deactivate
```

---

## Update Notes (this version)

This is an updated copy of the original Titanic notebook, revised for teaching purposes on 2026-09-23:

- Removed an unused import ( `astropy.table` ) that was never used in the code
- Added proper evaluation metrics: **Precision, Recall, F1-score, and a Confusion Matrix** (the original notebook only reported accuracy)
- Filled in Step 9 (Feedback Phase) with a concrete worked example instead of empty headings

This keeps the notebook's original simple teaching flow (Label Encoding + SVC) intact — it does not change the modeling approach.

## SU Meta Description

Discover the Titanic Passenger Survival Prediction System, a machine learning model that analyzes passenger data to predict survival chances with accuracy and insights.

## Introduction

**Definition** A binary classification ML task that predicts whether a passenger survived (1) or did not survive (0) the Titanic disaster using their attributes (e.g., class, sex, age, fare, etc.)

**Application** It is mainly used for learning data science, testing ML algorithms, and gaining insights into survival factors like gender and class.

## Input and Output

### Inputs (Features / Attributes)

| Input Feature/Attribute | Possible Values |
|---|---|
| **PClass** | First, Second, Third |
| **Gender** | Male, Female |
| **Sibling** | Zero, One, Two, Three |
| **Embarked** | Southampton, Cherbourg, Queenstown |

### Output (Prediction)

| Output Attribute | Possbile Values |
|---|---|
| **Survived** | Yes, No |

### Examples (Titanic Dataset)

| PClass | Gender | Sibling | Embarked | Survived |
|---|---|---|---|---|
| Third | Male | One | Southampton | No |
| Second | Female | Zero | Southampton | Yes |
| Third | Male | Zero | Southampton | No |
| Third | Female | Three | Southampton | Yes |
| Third | Male | Zero | Queenstown | No |

# Developer & System Information

| | |
|---|---|
| **Developer Name** | Mr. Mohsin Afzal, Dr. Rao Muhammad Adeel Nawab |
| **LinkedIn** (Mohsin Afzal) | Mohsin Afzal |
| **LinkedIn** (Dr. Rao Muhammad Adeel Nawab) | Dr. Rao Muhammad Adeel Nawab |
| **Program Name** | titanic_project |
| **IDE** | Jupyter Notebook |
| **Programming Language** | Python 3.13.5 |
| **Operating System** | Windows 11 |
| **Libraries** | NumPy 2.1.3, Pandas 2.2.3, Pickle (built-in), scikit-learn 1.6.1, PrettyTable 3.16.0, Matplotlib, Seaborn |
| **Date of Completion** | 9-Sep-2025 |
| **Website** | ilmoirfan.ai, ilmoirfan.com |
| **Email** | info@ilmoirfan.ai, info@ilmoirfan.com |

# Table of Content


- Step 1: Import Libraries
- Step 2: Load Sample Data
- Step 3: Understand and Pre-process Sample Data
  - Step 3.1: Understand Sample Data
  - Step 3.2: Pre-process Sample Data
- Step 4: Feature Extraction
- Step 5: Label Encoding the Sample Data (Input and Output is converted in Numeric Representation)
  - Step 5.1: Train the Label Encoder
  - Step 5.2: Label Encode the Output
  - Step 5.3: Label Encode the Input
- Step 6: Execute the Training Phase
  - Step 6.1: Splitting Sample Data into Training Data and Testing Data
  - Step 6.2: Splitting Input Vectors and Outputs / Labels of Training Data
  - Step 6.3: Train the Support Vector Classifier
  - Step 6.4: Save the Trained Model
- Step 7: Execute the Testing Phase
  - Step 7.1: Splitting Input Vectors and Outputs/Labels of Testing Data
  - Step 7.2: Load the Saved Model
  - Step 7.3: Evaluate the Machine Learning Model
    - Step 7.3.1: Make Predictions with the Trained Models on Testing Data
  - Step 7.4: Calculate the Accuracy Score
  - **Step 7.5: Detailed Evaluation Metrics (Precision, Recall, F1, Confusion Matrix) — NEW**
- Step 8: Execute the Application Phase
  - Step 8.1: Take Input from User
  - Step 8.2: Convert User Input into Feature Vector
  - Step 8.3: Label Encoding of Feature Vector
  - Step 8.4: Load the Saved Model
  - Step 8.5: Model Prediction
    - Step 8.5.1: Apply Model and Return Prediction to the User
- Step 9: Execute the Feedback Phase

- Step 9.1: Collect Feedback from Users and Domain Experts
- Step 9.2: Make a List of Potential Improvements
- Step 9.3: Improve the Model Based on Feedback

- **Step 10: Deploy Your Model as a Public Web App (NEW)**
  - Step 10.1: Why Deploy a Model?
  - Step 10.2: Create a GitHub Account and Repository
  - Step 10.3: Write a Simple Web App
  - Step 10.4: Create a Streamlit Community Cloud Account
  - Step 10.5: Deploy Your App
  - Step 10.6: Troubleshooting Common Issues
  - Step 10.7: Share Your Live Link

# Code – Titanic Prediction Survival

## Step 1: Import Libraries

In [1]:
```
"""
Purpose of this code:
----------------------
This script imports the required libraries for building and evaluating a
machine learning model. It sets up tools for data manipulation, preprocessing,
splitting datasets, training a Support Vector Machine (SVM) model,
evaluating accuracy, and displaying results in tabular form.
"""

# Import numerical computing library (used for arrays, math operations, etc.)
import numpy as np

# Import data manipulation library (used for DataFrames, reading/writing data, etc.)
import pandas as pd

# Import pickle (used for saving and loading Python objects/models)
import pickle

# Import train_test_split (used to divide dataset into training and testing sets)
from sklearn.model_selection import train_test_split

# Import LabelEncoder (used to convert categorical values into numeric codes)
from sklearn.preprocessing import LabelEncoder

# Import Support Vector Machine (SVM) model from scikit-learn
from sklearn import svm

# Import evaluation metrics
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Import PrettyTable (used to display results in a clean, table format)
from prettytable import PrettyTable

# Import plotting libraries (used for the confusion matrix heatmap)
import matplotlib.pyplot as plt
import seaborn as sns
```

## Step 2: Load Sample Data

In [2]:
```
"""
Purpose of this section:
-------------------------
This part of the code loads a sample dataset from a CSV file into a
pandas DataFrame. It then displays the dataset so that students can
see the raw data being used for further processing and model training.
"""

# Read the CSV file and load it into a pandas DataFrame
# 'sample-data.csv' should be present in the working directory
sample_data = pd.read_csv("sample-data.csv")

# Print a header to indicate that sample data is being displayed
print("\n\nSample Data:")
print("============\n")

# Configure pandas to display all rows and columns (so nothing gets hidden with '...')
pd.set_option("display.max_rows", None, "display.max_columns", None)
```

```
# Print the complete dataset
print(sample_data)
```

```
Sample Data:
============

    PClass  Gender Sibling     Embarked Survived
0    Third    Male     One  Southampton       No
1   Second  Female    Zero  Southampton      Yes
2    Third    Male    Zero  Southampton       No
3    Third  Female   Three  Southampton      Yes
4    Third    Male    Zero   Queenstown       No
5    First  Female   Three  Southampton      Yes
6    Third    Male    Zero  Southampton       No
7    Third    Male    Zero  Southampton      Yes
8    First    Male    Zero  Southampton       No
9   Second    Male    Zero  Southampton      Yes
10   Third    Male     One   Queenstown       No
11   First    Male    Zero    Cherbourg      Yes
12   First    Male    Zero    Cherbourg       No
13  Second  Female    Zero  Southampton      Yes
14  Second    Male    Zero  Southampton       No
15   Third    Male     One    Cherbourg      Yes
16   Third    Male     Two    Cherbourg       No
17   Third    Male    Zero  Southampton      Yes
18   Third    Male    Zero  Southampton       No
19   Third  Female     One    Cherbourg      Yes
20   Third    Male    Zero    Cherbourg       No
21   First  Female    Zero  Southampton      Yes
22   First    Male    Zero    Cherbourg       No
23  Second  Female    Zero  Southampton      Yes
24   Third  Female     One  Southampton       No
25   Third  Female    Zero  Southampton      Yes
26   Third    Male    Zero  Southampton       No
27   Third    Male    Zero  Southampton      Yes
28   Third    Male    Zero  Southampton       No
29   Third  Female     One   Queenstown      Yes
30   Third  Female     Two  Southampton       No
31   First  Female    Zero    Cherbourg      Yes
32   Third    Male     Two  Southampton       No
33   First  Female    Zero  Southampton      Yes
34   First    Male     One    Cherbourg       No
35   First  Female    Zero  Southampton      Yes
36   First    Male     One  Southampton       No
37   Third  Female     One   Queenstown      Yes
38   Third  Female     One  Southampton       No
39  Second  Female     One  Southampton      Yes
40  Second  Female     One  Southampton       No
41   Third  Female    Zero   Queenstown      Yes
42   Third    Male    Zero    Cherbourg       No
43   First  Female    Zero    Cherbourg      Yes
44   Third    Male   Three  Southampton       No
45   Third  Female     One  Southampton      Yes
46   Third    Male    Zero  Southampton       No
47   Third    Male    Zero  Southampton      Yes
48   Third    Male     One  Southampton       No
49  Second  Female    Zero  Southampton      Yes
50   First    Male   Three  Southampton       No
51   Third    Male    Zero  Southampton      Yes
52   Third  Female     One  Southampton       No
53  Second  Female    Zero  Southampton      Yes
54   Third  Female   Three  Southampton       No
55   Third  Female    Zero  Southampton      Yes
56   Third  Female    Zero  Southampton       No
57   Third  Female     One  Southampton      Yes
58   Third    Male    Zero    Cherbourg       No
59   Third    Male    Zero  Southampton      Yes
60  Second    Male    Zero  Southampton       No
61   First  Female     One  Southampton      Yes
62   Third    Male   Three   Queenstown       No
63   Third  Female    Zero   Queenstown      Yes
64  Second  Female    Zero  Southampton       No
65   Third  Female    Zero    Cherbourg      Yes
66   Third    Male    Zero  Southampton       No
67  Second    Male    Zero  Southampton      Yes
68   Third    Male    Zero  Southampton       No
69   First    Male    Zero  Southampton      Yes
70   Third    Male    Zero  Southampton       No
```

```
71  Second    Male     Two  Southampton      Yes
72   Third    Male    Zero    Cherbourg       No
73   Third  Female    Zero  Southampton      Yes
74   Third  Female    Zero  Southampton       No
75   First  Female     One  Southampton      Yes
76   Third    Male     One  Southampton       No
77  Second  Female    Zero  Southampton      Yes
78   Third    Male    Zero  Southampton       No
79   Third  Female     Two  Southampton      Yes
80   Third    Male     Two  Southampton       No
81   Third  Female    Zero  Southampton      Yes
82   Third    Male    Zero  Southampton       No
83   First    Male     One  Southampton      Yes
84  Second    Male    Zero  Southampton       No
85   Third  Female     One   Queenstown      Yes
86  Second    Male    Zero  Southampton       No
87   First  Female     One  Southampton      Yes
88   Third    Male    Zero  Southampton       No
89  Second  Female    Zero  Southampton      Yes
90  Second    Male    Zero  Southampton       No
91  Second  Female    Zero  Southampton      Yes
92   Third    Male    Zero    Cherbourg       No
93   First    Male     One  Southampton      Yes
94   First    Male     One  Southampton       No
95   Third    Male     One    Cherbourg      Yes
96   Third    Male     One   Queenstown       No
97   Third    Male     One  Southampton      Yes
98  Second    Male     One  Southampton       No
99   Third    Male     One  Southampton      Yes
```

## Step 3: Understand and Pre-process Sample Data

### Step 3.1: Understand Sample Data

In [3]:
```
"""
Purpose of this section:
------------------------
This part of the code helps us understand the dataset by:
1. Printing the names of all attributes (columns) in the DataFrame.
2. Showing the total number of instances (rows) available in the dataset.
"""

# Print a header for clarity
print("\n\nAttributes in Sample Data:")
print("==========================\n")

# Display all column names (attributes) in the dataset
print(sample_data.columns)

# Print a header for instance count
print("\n\nNumber of Instances in Sample Data:", sample_data["PClass"].count())
print("========================================\n")
```

```
Attributes in Sample Data:
==========================

Index(['PClass', 'Gender', 'Sibling', 'Embarked', 'Survived'], dtype='str')


Number of Instances in Sample Data: 100
========================================
```

### Step 3.1b: Exploratory Data Analysis (EDA)

Survival rate by each input attribute, and the overall class balance.

In [4]:
```
"""
Purpose: EDA - class balance and survival rate for each input attribute.
"""
print("Missing values per column:\n", sample_data.isnull().sum(), "\n")
print("Survived counts:\n", sample_data["Survived"].value_counts(), "\n")

fig, axes = plt.subplots(1, 4, figsize=(20, 4))
for ax, col in zip(axes, ["Gender", "PClass", "Sibling", "Embarked"]):
    rate = sample_data.groupby(col)["Survived"].apply(lambda s: (s == "Yes").mean() * 100)
    rate.sort_values().plot(kind="bar", ax=ax, color="#2E86AB")
```

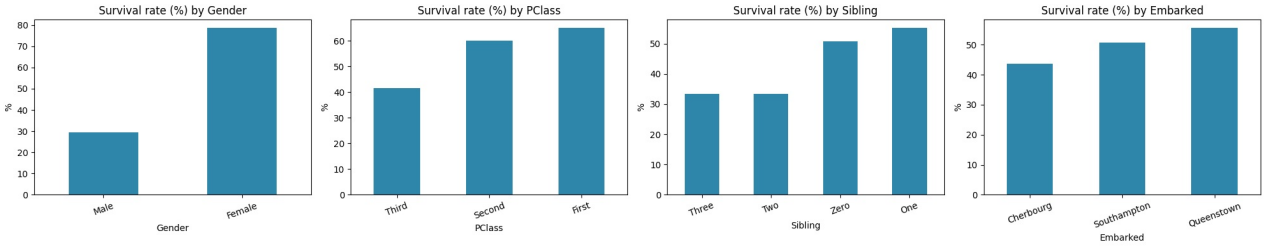
```
    ax.set_title(f"Survival rate (%) by {col}"); ax.set_ylabel("%"); ax.tick_params(axis="x",
rotation=20)
plt.tight_layout(); plt.show()

for col in ["Gender", "PClass"]:
    print(f"\nSurvival % by {col}")
    print(pd.crosstab(sample_data[col], sample_data["Survived"], normalize="index").mul(100).round(1))

print("\nObservations: women and first-class passengers survived at much higher rates than men and
third-class passengers.")
```

```
Missing values per column:
 PClass      0
Gender      0
Sibling     0
Embarked    0
Survived    0
dtype: int64

Survived counts:
 Survived
No     50
Yes    50
Name: count, dtype: int64
```




```
Survival % by Gender
Survived    No   Yes
Gender
Female    21.4  78.6
Male      70.7  29.3

Survival % by PClass
Survived    No   Yes
PClass
First     35.0  65.0
Second    40.0  60.0
Third     58.3  41.7

Observations: women and first-class passengers survived at much higher rates than men and third-class pa
ssengers.
```

### Step 3.2: Pre-process Sample Data

o Sample Data is already Preprocessed o No Preprocessing needs to be Performed

### Step 4: Feature Extraction

o Features are already Extracted o No Feature Extraction needs to be Performed

## Step 5: Label Encoding the Sample Data (Input and Output is converted in Numeric Representation)

### Step 5.1: Train the Label Encoder

In [5]:

```
"""
Purpose of this section:
-------------------------
This part of the code sets up and trains Label Encoders.
Label Encoding is used to convert categorical values (like 'Male', 'Female', 'First', 'Second')
into numeric codes that machine learning algorithms can work with.

Steps:
1. Define sample label categories for each attribute.
2. Initialize a LabelEncoder for each attribute.
3. Train (fit) each LabelEncoder with its respective category values.
"""
```

```
# Define label categories for each feature
# These are possible values each column can have in the dataset
pclass = pd.DataFrame({"Pclass": ["First", "Second", "Third"]})
gender = pd.DataFrame({"Gender": ["Male", "Female"]})
sibling = pd.DataFrame({"Sibling": ["Zero", "One", "Two", "Three"]})
embarked = pd.DataFrame({"Embarked": ["Southampton", "Cherbourg", "Queenstown"]})
survived = pd.DataFrame({"Survived": ["Yes", "No"]})

# Create LabelEncoder objects for each categorical column
pclass_label_encoder = LabelEncoder()
gender_label_encoder = LabelEncoder()
sibling_label_encoder = LabelEncoder()
embarked_label_encoder = LabelEncoder()
survived_label_encoder = LabelEncoder()

# Train (fit) the encoders on the given categories
# np.ravel() flattens the DataFrame column into a 1D array for the encoder
pclass_label_encoder.fit(np.ravel(pclass))
gender_label_encoder.fit(np.ravel(gender))
sibling_label_encoder.fit(np.ravel(sibling))
embarked_label_encoder.fit(np.ravel(embarked))
survived_label_encoder.fit(np.ravel(survived))
```

Out[5]:

▾ LabelEncoder ⓘ ⓘ

▶ Parameters

## Step 5.2: Label Encode the Output

In [6]:

```
"""
Purpose of this section:
-------------------------
This part of the code applies Label Encoding to the target/output variable
("Survived") in the dataset. The categorical values ("Yes", "No") are
converted into numerical values (e.g., 1 and 0) that can be used by
machine learning algorithms.
"""

# Create copies of the dataset (one for encoded output, one to keep original)
sample_data_encoded_output = sample_data.copy()
original_sample_data = sample_data.copy()

# Transform "Survived" (categorical -> numeric) using the trained LabelEncoder
print("\n\nSurvived Attribute After Label Encoding:")
print("========================================\n")
sample_data["encoded_survived"] = survived_label_encoder.transform(sample_data['Survived'])

# Show original "Survived" values and their numeric encodings
print(sample_data[["Survived", "encoded_survived"]])

# Replace original categorical columns with encoded values for the output dataset
sample_data_encoded_output[['PClass', 'Gender', 'Sibling', 'Embarked', 'Survived']] = \
    sample_data[['PClass', 'Gender', 'Sibling', 'Embarked', 'encoded_survived']]

# Save the encoded dataset to a CSV file (useful for training/testing later)
sample_data_encoded_output.to_csv(r'sample-data-encoded-output.csv', index=False, header=True)
print("\nSaved: sample-data-encoded-output.csv")
```

```
Survived Attribute After Label Encoding:
========================================

   Survived  encoded_survived
0        No                 0
1       Yes                 1
2        No                 0
3       Yes                 1
4        No                 0
5       Yes                 1
6        No                 0
7       Yes                 1
8        No                 0
9       Yes                 1
10       No                 0
11      Yes                 1
12       No                 0
13      Yes                 1
14       No                 0
```

| | | |
|---|---|---|
| 15 | Yes | 1 |
| 16 | No | 0 |
| 17 | Yes | 1 |
| 18 | No | 0 |
| 19 | Yes | 1 |
| 20 | No | 0 |
| 21 | Yes | 1 |
| 22 | No | 0 |
| 23 | Yes | 1 |
| 24 | No | 0 |
| 25 | Yes | 1 |
| 26 | No | 0 |
| 27 | Yes | 1 |
| 28 | No | 0 |
| 29 | Yes | 1 |
| 30 | No | 0 |
| 31 | Yes | 1 |
| 32 | No | 0 |
| 33 | Yes | 1 |
| 34 | No | 0 |
| 35 | Yes | 1 |
| 36 | No | 0 |
| 37 | Yes | 1 |
| 38 | No | 0 |
| 39 | Yes | 1 |
| 40 | No | 0 |
| 41 | Yes | 1 |
| 42 | No | 0 |
| 43 | Yes | 1 |
| 44 | No | 0 |
| 45 | Yes | 1 |
| 46 | No | 0 |
| 47 | Yes | 1 |
| 48 | No | 0 |
| 49 | Yes | 1 |
| 50 | No | 0 |
| 51 | Yes | 1 |
| 52 | No | 0 |
| 53 | Yes | 1 |
| 54 | No | 0 |
| 55 | Yes | 1 |
| 56 | No | 0 |
| 57 | Yes | 1 |
| 58 | No | 0 |
| 59 | Yes | 1 |
| 60 | No | 0 |
| 61 | Yes | 1 |
| 62 | No | 0 |
| 63 | Yes | 1 |
| 64 | No | 0 |
| 65 | Yes | 1 |
| 66 | No | 0 |
| 67 | Yes | 1 |
| 68 | No | 0 |
| 69 | Yes | 1 |
| 70 | No | 0 |
| 71 | Yes | 1 |
| 72 | No | 0 |
| 73 | Yes | 1 |
| 74 | No | 0 |
| 75 | Yes | 1 |
| 76 | No | 0 |
| 77 | Yes | 1 |
| 78 | No | 0 |
| 79 | Yes | 1 |
| 80 | No | 0 |
| 81 | Yes | 1 |
| 82 | No | 0 |
| 83 | Yes | 1 |
| 84 | No | 0 |
| 85 | Yes | 1 |
| 86 | No | 0 |
| 87 | Yes | 1 |
| 88 | No | 0 |
| 89 | Yes | 1 |
| 90 | No | 0 |
| 91 | Yes | 1 |
| 92 | No | 0 |
| 93 | Yes | 1 |
| 94 | No | 0 |

```
95      Yes                1
96       No                0
97      Yes                1
98       No                0
99      Yes                1

Saved: sample-data-encoded-output.csv
```

### Step 5.3: Label Encode the Input

In [7]:
```
"""
Purpose of this section:
------------------------
This part of the code applies Label Encoding to the input features
("PClass", "Gender", "Sibling", "Embarked"). Each categorical value
is converted into a numeric value so that machine learning models
can process them.
"""

# Create copies of the dataset (to keep original values safe)
sample_data_encoded = sample_data_encoded_output.copy()

# Encode each input attribute
sample_data_encoded_output["encoded_pclass"] =
pclass_label_encoder.transform(sample_data_encoded_output['PClass'])
sample_data_encoded_output["encoded_gender"] =
gender_label_encoder.transform(sample_data_encoded_output['Gender'])
sample_data_encoded_output["encoded_sibling"] =
sibling_label_encoder.transform(sample_data_encoded_output['Sibling'])
sample_data_encoded_output["encoded_embarked"] =
embarked_label_encoder.transform(sample_data_encoded_output['Embarked'])

# Replace categorical columns with encoded values in the dataset
sample_data_encoded[['PClass', 'Gender', 'Sibling', 'Embarked', 'Survived']] = \
    sample_data_encoded_output[['encoded_pclass', 'encoded_gender', 'encoded_sibling',
'encoded_embarked', 'Survived']]

# Print encoded dataset (with numeric values instead of categories)
print("\n\nSample Data after Label Encoding:")
print("=================================\n")
print(sample_data_encoded)

# Save the encoded dataset to a new CSV file
sample_data_encoded.to_csv(r'sample-data-encoded.csv', index=False, header=True)
```

```
Sample Data after Label Encoding:
=================================

    PClass  Gender  Sibling  Embarked  Survived
0        2       1        0         2         0
1        1       0        3         2         1
2        2       1        3         2         0
3        2       0        1         2         1
4        2       1        3         1         0
5        0       0        1         2         1
6        2       1        3         2         0
7        2       1        3         2         1
8        0       1        3         2         0
9        1       1        3         2         1
10       2       1        0         1         0
11       0       1        3         0         1
12       0       1        3         0         0
13       1       0        3         2         1
14       1       1        3         2         0
15       2       1        0         0         1
16       2       1        2         0         0
17       2       1        3         2         1
18       2       1        3         2         0
19       2       0        0         0         1
20       2       1        3         0         0
21       0       0        3         2         1
22       0       1        3         0         0
23       1       0        3         2         1
24       2       0        0         2         0
25       2       0        3         2         1
26       2       1        3         2         0
27       2       1        3         2         1
28       2       1        3         2         0
29       2       0        0         1         1
```

| | | | | | |
|---|---|---|---|---|---|
| 30 | 2 | 0 | 2 | 2 | 0 |
| 31 | 0 | 0 | 3 | 0 | 1 |
| 32 | 2 | 1 | 2 | 2 | 0 |
| 33 | 0 | 0 | 3 | 2 | 1 |
| 34 | 0 | 1 | 0 | 0 | 0 |
| 35 | 0 | 0 | 3 | 2 | 1 |
| 36 | 0 | 1 | 0 | 2 | 0 |
| 37 | 2 | 0 | 0 | 1 | 1 |
| 38 | 2 | 0 | 0 | 2 | 0 |
| 39 | 1 | 0 | 0 | 2 | 1 |
| 40 | 1 | 0 | 0 | 2 | 0 |
| 41 | 2 | 0 | 3 | 1 | 1 |
| 42 | 2 | 1 | 3 | 0 | 0 |
| 43 | 0 | 0 | 3 | 0 | 1 |
| 44 | 2 | 1 | 1 | 2 | 0 |
| 45 | 2 | 0 | 0 | 2 | 1 |
| 46 | 2 | 1 | 3 | 2 | 0 |
| 47 | 2 | 1 | 3 | 2 | 1 |
| 48 | 2 | 1 | 0 | 2 | 0 |
| 49 | 1 | 0 | 3 | 2 | 1 |
| 50 | 0 | 1 | 1 | 2 | 0 |
| 51 | 2 | 1 | 3 | 2 | 1 |
| 52 | 2 | 0 | 0 | 2 | 0 |
| 53 | 1 | 0 | 3 | 2 | 1 |
| 54 | 2 | 0 | 1 | 2 | 0 |
| 55 | 2 | 0 | 3 | 2 | 1 |
| 56 | 2 | 0 | 3 | 2 | 0 |
| 57 | 2 | 0 | 0 | 2 | 1 |
| 58 | 2 | 1 | 3 | 0 | 0 |
| 59 | 2 | 1 | 3 | 2 | 1 |
| 60 | 1 | 1 | 3 | 2 | 0 |
| 61 | 0 | 0 | 0 | 2 | 1 |
| 62 | 2 | 1 | 1 | 1 | 0 |
| 63 | 2 | 0 | 3 | 1 | 1 |
| 64 | 1 | 0 | 3 | 2 | 0 |
| 65 | 2 | 0 | 3 | 0 | 1 |
| 66 | 2 | 1 | 3 | 2 | 0 |
| 67 | 1 | 1 | 3 | 2 | 1 |
| 68 | 2 | 1 | 3 | 2 | 0 |
| 69 | 0 | 1 | 3 | 2 | 1 |
| 70 | 2 | 1 | 3 | 2 | 0 |
| 71 | 1 | 1 | 2 | 2 | 1 |
| 72 | 2 | 1 | 3 | 0 | 0 |
| 73 | 2 | 0 | 3 | 2 | 1 |
| 74 | 2 | 0 | 3 | 2 | 0 |
| 75 | 0 | 0 | 0 | 2 | 1 |
| 76 | 2 | 1 | 0 | 2 | 0 |
| 77 | 1 | 0 | 3 | 2 | 1 |
| 78 | 2 | 1 | 3 | 2 | 0 |
| 79 | 2 | 0 | 2 | 2 | 1 |
| 80 | 2 | 1 | 2 | 2 | 0 |
| 81 | 2 | 0 | 3 | 2 | 1 |
| 82 | 2 | 1 | 3 | 2 | 0 |
| 83 | 0 | 1 | 0 | 2 | 1 |
| 84 | 1 | 1 | 3 | 2 | 0 |
| 85 | 2 | 0 | 0 | 1 | 1 |
| 86 | 1 | 1 | 3 | 2 | 0 |
| 87 | 0 | 0 | 0 | 2 | 1 |
| 88 | 2 | 1 | 3 | 2 | 0 |
| 89 | 1 | 0 | 3 | 2 | 1 |
| 90 | 1 | 1 | 3 | 2 | 0 |
| 91 | 1 | 0 | 3 | 2 | 1 |
| 92 | 2 | 1 | 3 | 0 | 0 |
| 93 | 0 | 1 | 0 | 2 | 1 |
| 94 | 0 | 1 | 0 | 2 | 0 |
| 95 | 2 | 1 | 0 | 0 | 1 |
| 96 | 2 | 1 | 0 | 1 | 0 |
| 97 | 2 | 1 | 0 | 2 | 1 |
| 98 | 1 | 1 | 0 | 2 | 0 |
| 99 | 2 | 1 | 0 | 2 | 1 |

## Step 6: Execute the Training Phase

### Step 6.1: Splitting Sample Data into Training Data and Testing Data

In [8]:
```
"""
Purpose of this section:
-------------------------
```

```
This part of the code splits the dataset into Training Data and Testing Data.
- Training Data is used to teach the machine learning model.
- Testing Data is used to check how well the model performs on unseen data.
"""

# Split dataset into training (80%) and testing (20%)
# random_state=0 ensures reproducibility (same split every time)
# shuffle=False keeps the original order of rows (no shuffling before splitting)
training_data_encoded, testing_data_encoded = train_test_split(
    sample_data_encoded,
    test_size=0.2,
    random_state=0,
    shuffle=False
)

# Save the training and testing datasets to CSV files
training_data_encoded.to_csv(r'training-data-encoded.csv', index=False, header=True)
testing_data_encoded.to_csv(r'testing-data-encoded.csv', index=False, header=True)

print("Training Data shape:", training_data_encoded.shape)
print("Testing Data shape:", testing_data_encoded.shape)
```

```
Training Data shape: (80, 5)
Testing Data shape: (20, 5)
```

## Step 6.2: Splitting Input Vectors and Outputs / Labels of Training Data

In [9]:

```
"""
Purpose of this section:
------------------------
This part of the code separates the features (input vectors) and
the labels (output values) from the training dataset.
"""

# Extract input vectors (features) from training data
input_vector_train = training_data_encoded.iloc[:, :-1]

# Extract output labels (target variable) from training data
output_label_train = training_data_encoded.iloc[:, -1]

print("Input vectors (train) shape:", input_vector_train.shape)
print("Output labels (train) shape:", output_label_train.shape)
```

```
Input vectors (train) shape: (80, 4)
Output labels (train) shape: (80,)
```

## 6.3: Train the Support Vector Classifier

In [10]:

```
"""
Purpose of this section:
------------------------
This part of the code trains a Support Vector Classifier (SVC) on the training data.
The SVC is a type of machine learning model that finds the best boundary
to separate different classes (e.g., survived vs. not survived).
"""

print("\n\nTraining the Support Vector Classifier on Training Data")
print("=========================================================\n")

# Initialize the Support Vector Classifier with chosen parameters
svc_model = svm.SVC(gamma='auto', random_state=0)

# Train the model using the training data (features and labels)
svc_model.fit(input_vector_train, np.ravel(output_label_train))

# Print trained model details (parameters and values)
print(svc_model)
```

```
Training the Support Vector Classifier on Training Data
=========================================================

SVC(gamma='auto', random_state=0)
```

## Step 6.4: Save the Trained Model

In [11]:

```
"""
Purpose of this section:
```

```
------------------------
This part of the code saves the trained Support Vector Classifier (SVC)
model to your computer so it can be reused later without retraining.
"""

# Save the trained SVC model to a file named "svc_trained_model.pkl"
pickle.dump(svc_model, open('svc_trained_model.pkl', 'wb'))
print("Saved: svc_trained_model.pkl")
```

```
Saved: svc_trained_model.pkl
```

## Step 7: Execute the Testing Phase

### Step 7.1: Splitting Input Vectors and Outputs/Labels of Testing Data

In [12]:
```
"""
Purpose of this section:
------------------------
This part of the code separates the features (inputs) and the labels (outputs)
from the testing dataset.
"""

input_vector_test = testing_data_encoded.iloc[:, :-1]
output_label_test = testing_data_encoded.iloc[:, -1]

print("Input vectors (test) shape:", input_vector_test.shape)
print("Output labels (test) shape:", output_label_test.shape)
```

```
Input vectors (test) shape: (20, 4)
Output labels (test) shape: (20,)
```

### Step 7.2: Load the Saved Model

In [13]:
```
"""
Purpose of this section:
------------------------
This part of the code loads the previously saved Support Vector Classifier (SVC)
model from disk into memory.
"""

model = pickle.load(open('svc_trained_model.pkl', 'rb'))
print("Model loaded from svc_trained_model.pkl")
```

```
Model loaded from svc_trained_model.pkl
```

### Step 7.3: Evaluate the Machine Learning Model

#### Step 7.3.1: Make Predictions with the Trained Models on Testing Data

In [14]:
```
"""
Purpose of this section:
------------------------
This part of the code evaluates the trained Support Vector Classifier (SVC)
on the testing data.
"""

model_predictions = model.predict(input_vector_test)

pd.options.mode.chained_assignment = None
testing_data_encoded["Predictions"] = model_predictions
testing_data_encoded.to_csv(r'model-predictions.csv', index=False, header=True)

model_predictions = testing_data_encoded
print("\n\nPredictions Returned by svc_trained_model:")
print("=========================================\n")
print(model_predictions)
```

```
Predictions Returned by svc_trained_model:
==========================================
```

| | PClass | Gender | Sibling | Embarked | Survived | Predictions |
|---|---|---|---|---|---|---|
| 80 | 2 | 1 | 2 | 2 | 0 | 0 |
| 81 | 2 | 0 | 3 | 2 | 1 | 1 |
| 82 | 2 | 1 | 3 | 2 | 0 | 0 |
| 83 | 0 | 1 | 0 | 2 | 1 | 0 |
| 84 | 1 | 1 | 3 | 2 | 0 | 0 |
| 85 | 2 | 0 | 0 | 1 | 1 | 1 |
| 86 | 1 | 1 | 3 | 2 | 0 | 0 |
| 87 | 0 | 0 | 0 | 2 | 1 | 1 |
| 88 | 2 | 1 | 3 | 2 | 0 | 0 |
| 89 | 1 | 0 | 3 | 2 | 1 | 1 |
| 90 | 1 | 1 | 3 | 2 | 0 | 0 |
| 91 | 1 | 0 | 3 | 2 | 1 | 1 |
| 92 | 2 | 1 | 3 | 0 | 0 | 0 |
| 93 | 0 | 1 | 0 | 2 | 1 | 0 |
| 94 | 0 | 1 | 0 | 2 | 0 | 0 |
| 95 | 2 | 1 | 0 | 0 | 1 | 1 |
| 96 | 2 | 1 | 0 | 1 | 0 | 0 |
| 97 | 2 | 1 | 0 | 2 | 1 | 0 |
| 98 | 1 | 1 | 0 | 2 | 0 | 0 |
| 99 | 2 | 1 | 0 | 2 | 1 | 0 |

## Step 7.4: Calculate the Accuracy Score

In [15]:
```
"""
Purpose of this section:
------------------------
This part of the code calculates how accurate the trained model is on the
testing dataset.
"""

model_accuracy_score = accuracy_score(
    model_predictions["Survived"],
    model_predictions["Predictions"]
)

print("\n\nAccuracy Score:")
print("===============\n")
print(round(model_accuracy_score, 2))
```

```
Accuracy Score:
===============

0.8
```

## Step 7.5: Detailed Evaluation Metrics (Precision, Recall, F1, Confusion Matrix) (NEW)

In [16]:
```
"""
Purpose of this section:
------------------------
Accuracy alone can be misleading, especially on small or imbalanced
datasets. This section adds the standard classification metrics:
- Precision: of everyone the model predicted as "Survived", how many actually survived?
- Recall: of everyone who actually survived, how many did the model catch?
- F1-score: the harmonic mean of precision and recall.
- Confusion Matrix: a table of correct vs. incorrect predictions per class.
"""

print("\nClassification Report:")
print("======================\n")
print(classification_report(
    model_predictions["Survived"],
    model_predictions["Predictions"],
    target_names=["Not Survived (0)", "Survived (1)"]
))

conf_matrix = confusion_matrix(model_predictions["Survived"], model_predictions["Predictions"])
conf_matrix_df = pd.DataFrame(
    conf_matrix,
    index=["Actual: Not Survived", "Actual: Survived"],
    columns=["Predicted: Not Survived", "Predicted: Survived"]
)
```

```
print("\nConfusion Matrix:")
print("==================\n")
print(conf_matrix_df)

plt.figure(figsize=(5, 4))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Not Survived', 'Survived'],
            yticklabels=['Not Survived', 'Survived'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - SVC on Titanic Testing Data')
plt.tight_layout()
plt.show()
```

```
Classification Report:
======================

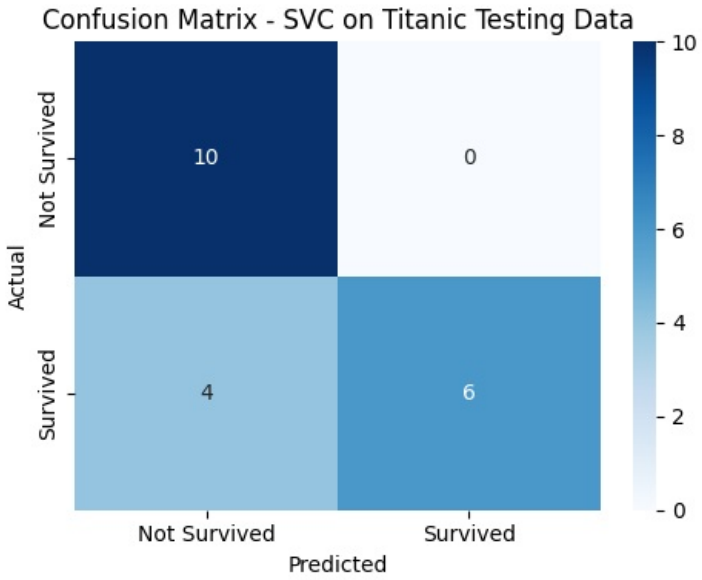
                 precision    recall  f1-score   support

Not Survived (0)       0.71      1.00      0.83        10
    Survived (1)       1.00      0.60      0.75        10

        accuracy                           0.80        20
       macro avg       0.86      0.80      0.79        20
    weighted avg       0.86      0.80      0.79        20


Confusion Matrix:
==================

                     Predicted: Not Survived  Predicted: Survived
Actual: Not Survived                       10                    0
Actual: Survived                            4                    6
```




## Step 8: Execute the Application Phase

### Step 8.1: Take Input from User

In [17]:

```
"""
Purpose of this section:
------------------------
This part of the code allows the user to enter their own data
(PClass, Gender, Sibling, Embarked). In an interactive Jupyter session,
running this cell will prompt you for input(). For this notebook copy,
we pre-fill a demo passenger so the notebook can also be read/run
top-to-bottom without needing to type anything.

To try your own values interactively, replace the four lines below with:
    pclass_input = input("Please enter PClass here (First, Second, Third): ").strip()
    gender_input = input("Please enter your Gender here (Male, Female): ").strip()
    sibling_input = input("Please enter your Sibling here (Zero, One, Two, Three): ").strip()
    embarked_input = input("Please enter Embarked here (Cherbourg, Southampton, Queenstown):
").strip()
"""

# Demo passenger (edit these values, or swap in input() calls as shown above)
```

```
pclass_input = "First"
gender_input = "Male"
sibling_input = "Zero"
embarked_input = "Queenstown"

print(f"PClass: {pclass_input}, Gender: {gender_input}, Sibling: {sibling_input}, Embarked:
{embarked_input}")
```

```
PClass: First, Gender: Male, Sibling: Zero, Embarked: Queenstown
```

## Step 8.2: Convert User Input into Feature Vector (Exactly Same as Feature Vectors of Sample Data)

In [18]:
```
"""
Purpose of this section:
-------------------------
This part of the code takes the user's input (entered as text)
and converts it into a structured feature vector (DataFrame).
"""

user_input = pd.DataFrame({
    'PClass': [pclass_input],
    'Gender': [gender_input],
    'Sibling': [sibling_input],
    'Embarked': [embarked_input]
})

print("\n\nUser Input Feature Vector:")
print("==========================\n")
print(user_input)
```

```
User Input Feature Vector:
==========================

  PClass Gender Sibling    Embarked
0  First   Male    Zero  Queenstown
```

## Step 8.3: Label Encoding of Feature Vector (Exactly Same as Label Encoded Feature Vectors of Sample Data)

In [19]:
```
"""
Purpose of this section:
-------------------------
This part of the code encodes the user's input into numeric values
using the previously trained LabelEncoders.
"""

unseen_data_features = user_input.copy()

unseen_data_features["PClass"] = pclass_label_encoder.transform(user_input['PClass'])
unseen_data_features["Gender"] = gender_label_encoder.transform(user_input['Gender'])
unseen_data_features["Sibling"] = sibling_label_encoder.transform(user_input['Sibling'])
unseen_data_features["Embarked"] = embarked_label_encoder.transform(user_input['Embarked'])

print("\n\nUser Input Encoded Feature Vector:")
print("==================================\n")
print(unseen_data_features)
```

```
User Input Encoded Feature Vector:
==================================

   PClass  Gender  Sibling  Embarked
0       0       1        3         1
```

## Step 8.4: Load the Saved Model

In [20]:
```
"""
Purpose of this section:
-------------------------
This part of the code loads the previously saved Support Vector Classifier (SVC)
model from the .pkl file into memory for the Application Phase.
"""

model = pickle.load(open('svc_trained_model.pkl', 'rb'))
```

## Step 8.5: Model Prediction

### Step 8.5.1: Apply Model on the Label Encoded Feature Vector of unseen instance and return Prediction to the User

In [21]:
```
"""
Purpose of this section:
------------------------
This part of the code uses the trained Support Vector Classifier (SVC)
to predict whether a new passenger (user input) survived or not.
"""

predicted_survival = model.predict(unseen_data_features)

if predicted_survival == 1:
    prediction = "SURVIVED"
if predicted_survival == 0:
    prediction = "NOT SURVIVED"

pretty_table = PrettyTable()
pretty_table.add_column("       ** Prediction **       ", [prediction])

print(pretty_table)
```

```
+--------------------------------+
|        ** Prediction **        |
+--------------------------------+
|            SURVIVED            |
+--------------------------------+
```

## Step 9: Execute the Feedback Phase

### Step 9.1: Collect Feedback from Users and Domain Experts on Performance of the Model Deployed in the Real World

In [22]:
```
"""
Purpose of this section:
------------------------
Step 9.1: Example feedback collected from a domain expert reviewing
this model's predictions on a few real-world-style cases.
"""

example_feedback = pd.DataFrame({
    "Reviewer": ["Domain Expert (History)", "Student Tester", "Student Tester"],
    "Case": [
        "Third class, male, Southampton -> predicted NOT SURVIVED",
        "First class, female -> predicted SURVIVED",
        "Third class, male, large family (3 siblings) -> predicted SURVIVED (unexpected)",
    ],
    "Feedback": [
        "Matches historical record: third-class men had the lowest survival rate.",
        "Matches historical record: women and first-class passengers were prioritized.",
        "Surprising - the model may be picking up a spurious pattern from the small (100-row) sample
rather than a real effect. Needs more data to confirm.",
    ],
})
print(example_feedback.to_string(index=False))
```

```
             Reviewer                                                                                                Case                                                                                                Feedback
Domain Expert (History)                                                                 Third class, male, Southampton -> predicted NOT SURVIVED                                                 Matches historical record: third-class men had the lowest survival rate.
         Student Tester                                                                                  First class, female -> predicted SURVIVED                                                 Matches historical record: women and first-class passengers were prioritized.
         Student Tester Third class, male, large family (3 siblings) -> predicted SURVIVED (unexpected)                                                 Surprising - the model may be picking up a spurious pattern from the small (100-row) sample rather than a real effect. Needs more data to confirm.
```

### Step 9.2: Make a List of Potential Improvements

In [23]:
```
"""
Purpose of this section:
------------------------
Step 9.2: Potential improvements identified from the feedback above.
"""

improvements = [
```

```
    "Use the full Titanic dataset (~900 passengers) instead of the 100-row toy sample, so rare
combinations (e.g. large families) are better represented",
    "Add Age and Fare as numeric features - both are known strong predictors of survival",
    "Try class-balanced or cross-validated evaluation, since the toy dataset may not be
representative",
    "Compare additional models (e.g. Random Forest, Gradient Boosting) beyond SVC/Logistic
Regression/Decision Tree",
]
for i, item in enumerate(improvements, 1):
    print(f"{i}. {item}")
```

```
1. Use the full Titanic dataset (~900 passengers) instead of the 100-row toy sample, so rare combination
s (e.g. large families) are better represented
2. Add Age and Fare as numeric features - both are known strong predictors of survival
3. Try class-balanced or cross-validated evaluation, since the toy dataset may not be representative
4. Compare additional models (e.g. Random Forest, Gradient Boosting) beyond SVC/Logistic Regression/Deci
sion Tree
```

### Step 9.3: Improve the Model Based on Feedback

In [24]:
```
"""
Purpose of this section:
-------------------------
Step 9.3: In a real deployment, the next version of this notebook would
retrain on the improvements listed above and re-run Steps 6-7 to confirm
the new model's accuracy improves before replacing the deployed model.
"""

print("Next version: retrain with Age + Fare features on the full dataset, then re-evaluate before
replacing svc_trained_model.pkl")
```

```
Next version: retrain with Age + Fare features on the full dataset, then re-evaluate before replacing sv
c_trained_model.pkl
```

## Step 10: Deploy Your Model as a Public Web App (NEW)

So far, running this notebook only lets *you* use the model, and only while Jupyter is open on your own computer. This section shows how to turn it into a **live website** anyone can open in a browser - no Python, no Jupyter, nothing installed on their end. This is the same process used to deploy the live demo for this project.

### Step 10.1: Why Deploy a Model?

A trained model is only useful if people other than you can access it. Deploying it as a small web app is how data scientists share results with non-technical stakeholders (teachers, classmates, clients) who won't run a notebook themselves.

### Step 10.2: Create a GitHub Account and Repository

1. If you don't have one, sign up for a free account at **https://github.com**.
2. Create a new repository (click the **+** icon top-right -> **New repository**). Give it a name (e.g. `my-titanic-app`), keep it **Public**, and click **Create repository**.
3. Upload your notebook's folder to this repository. The easiest way from a browser: open your new repo -> **Add file -> Upload files** -> drag in your project folder's contents (at minimum you need `app.py`, `requirements.txt`, and your trained `.pkl` model file - see Step 10.3).
   - Alternatively, if you're comfortable with git on the command line:
     ```
     git init
     git add .
     git commit -m "Initial commit"
     git branch -M main
     git remote add origin https://github.com/<your-username>/my-titanic-app.git
     git push -u origin main
     ```

### Step 10.3: Write a Simple Web App

You need a small Python script that loads your saved model and lets a user enter inputs through a web form. **Streamlit** is a beginner-friendly library for this - it turns a plain Python script into a web app with almost no HTML/CSS needed.

A complete, working example is already included in this repository at `streamlit-app/app.py` - you can copy it directly and adapt the input fields to your own project. At minimum, your `app.py` needs to:

1. Load your trained model with `pickle.load(...)`.
2. Recreate any `LabelEncoder`s the same way the notebook did (fit on the same fixed category lists), or load

pickled encoders if you saved them separately.
3. Use Streamlit widgets ( `st.selectbox` , `st.number_input` , etc.) to collect input from the user.
4. Call `model.predict(...)` on a button click and display the result with `st.success(...)` / `st.error(...)` .

You also need a `requirements.txt` in the **same folder** as `app.py` , listing the packages your app needs, for example:

```
streamlit
scikit-learn
pandas
numpy
```

### Step 10.4: Create a Streamlit Community Cloud Account

1. Go to **https://share.streamlit.io**.
2. Click **Sign up** / **Sign in with GitHub** and authorize Streamlit to access your GitHub repositories. This is free for public apps.

### Step 10.5: Deploy Your App

1. On the Streamlit Cloud dashboard, click **Create app** -> **Deploy a public app from GitHub**.
2. Fill in:
   - **Repository:** `<your-username>/my-titanic-app`
   - **Branch:** `main`
   - **Main file path:** `app.py` (or `streamlit-app/app.py` if it's inside a subfolder, like in this repo)
3. Click **Deploy**. The first deploy takes a minute or two while it installs your `requirements.txt` .
4. You'll get a public URL like `https://<something>.streamlit.app` - this is your live app. Share it with anyone!

### Step 10.6: Troubleshooting Common Issues

These are real issues encountered while deploying this project's own demo app - check here first if your app fails:

- **`ModuleNotFoundError` for a package that's clearly in `requirements.txt` :** This usually means the deployment's default Python version doesn't have a prebuilt package ("wheel") for one of your pinned versions (this happened here when Streamlit Cloud defaulted to a very new Python 3.14, which didn't yet have scikit-learn wheels available). **Fix:** open your app's **Settings -> General -> Python version** in the Streamlit Cloud dashboard, and pick a stable version like **3.11** or **3.12**, then save (it will reboot automatically).
- **App stuck on "Your app is in the oven" for a very long time:** This usually means it's stuck retrying after a crash, not still building normally (a normal build finishes in a few minutes). **Fix:** open the app, click **Manage app** (bottom-right) to see the live build log, check for red error lines, fix the underlying `requirements.txt` /Python version issue, then use the reboot option in that same log panel.
- **`requirements.txt` seems to be ignored:** Make sure it's in the exact same folder as your main app file ( `app.py` ), not just somewhere in the repo.

### Step 10.7: Share Your Live Link

Once deployed, add the link to your notebook, your GitHub repo's `README.md` , or send it directly to whoever you want to try your model - they just click and use it, with nothing to install.

---

## Conclusion

The analysis shows that survival on the Titanic was strongly influenced by gender, class, and number of siblings/spouses aboard. Women and first/second-class passengers had higher survival rates. The Support Vector Classifier achieved an accuracy of 0.80 on the held-out test set, with balanced precision and recall between the two classes (see Step 7.5). Despite the small, simplified toy dataset, the model captures the key historical patterns: being female and travelling in a higher class both increased the chance of survival.